

Decode Ladder + Discount Tire Harvest: Session Proof Report

Branch feat/decode-ladder-goupc - Ladder plan Task 20 Step 3 - Generated 2026-07-09. Offline synthesis of all proof artifacts produced this session; no live provider calls were made to produce this report.

Verdict: SHIPPED. The decode ladder (corpus -> Go-UPC -> Fetch V2 -> GPT-5.5) is now reachable in production for the first time (it was structurally dead for public barcodes before this session's fix). The one severe safety defect the live proof surfaced - a 4-digit vendor part number auto-verified with a fabricated identity - was found, root-caused, fixed, and reverified: auto-count precision moved from 96.9% to 100% on the exact 60-code run that exposed it. In parallel, the Discount Tire harvest pipeline ran its full catalog and grew the tire corpus by 2,029 new GTINs at \$0 spend. The binding qa:revision gate (tsc, eslint, build, full Playwright E2E, Firebase tests, 12/12 QA bots) is GREEN on the final tree. Nothing has been pushed or deployed; that remains the owner's call.

What shipped, in one table

Item	Before this session	After this session
Decode ladder reachability	Go-UPC/FetchV2/GPT structurally unreachable for public barcodes (Blatfix OverView: terminal)	Reachable for public barcodes (Blatfix OverView: terminal)
Auto-count precision (60-code live run)	96.9% (31/32 correct)	100% (25/25 correct) after the 1225 fix
Wrong auto-counts in the run	1 (code 1225, fabricated Hungarian cranberry pad) (UpToSelf: report: fauces: article) note to needs_review	0
Tire corpus size	76,173 rows (pre-harvest)	+2,029 new GTINs merged (5,553 cross-source dupes kept)
Binding qa:revision gate	n/a (mid-session, red at points - see Section 2.3)	GREEN: tsc + eslint + build + full Playwright E2E + Firebase

2. The Decode Ladder

2.1 Rung order and each rung's literal decision rule

Spec v6 (owner-approved, commit c957423): local corpus -> Go-UPC -> Fetch V2 -> GPT-5.5, sequential, first-settle stops the ladder. Gemini is permanently out of the decode path. Each rung's actual, as-built behavior (not the aspirational plan text):

Rung	Trigger / query	Decision rule as built
1. Local corpus (\$0)	SQLite exact-barcode lookup against knowledge graph local DB (tires + retail tables)	resolved local DB (tires + retail tables) with zero network calls on any
Plan D (pre-ladder, public barcodes only)	ean_13/gtin_14 codes: parallel floor + retail unknown consensus DEERAWT Search of 150 public-ba	
2. Go-UPC (<= 15 lookups/proofs)	isPublicBarcode(code) && isValidCheckDigit(code) Gates only live paid lookup	resolved unknown consensus DEERAWT Search of 150 public-ba
3. Fetch V2 (Brave + Firecrawl Search + cheap-scrape discovery over the web)	Search + cheap-scrape discovery over the web	Resolved only if the exact code is found in real evidence (Evidence
4. GPT-5.5 (probe parity, ~\$0.39/word in report/day cap)	Real world cases (8/day cap) reports exactCodeFound + corroborate the 'opts/scr_wrapper' + test (owner order 202607)	

2.2 Task 20 waterfall - settled-by-stage per group (60 live codes, real .env.local keys)

Group A = 20 tire barcodes freshly sampled from the corpus at run time. Groups B/C = 40 codes hand-selected from e2e/fixtures/dryrun-codes.json, reconfirmed absent from BOTH corpora at selection time AND at run time (zero corpus growth in between).

Group	n	Settled-by breakdown	Status breakdown
A - corpus tires	20	corpus_exact_barcode 20	verified 20
B - retail, absent from corpora	20	goupc 7, single_source 4, fetchv2 2, none 7	verified 5, suggested 2, needs_review 13
C - hard tail, absent from corpora	20	gpt 9, none 6, goupc 3, corpus_exact_part_number 1, none 1	verified 12, suggested 3, needs_review 10

2.3 Per-rung verdict: WORKING / NOT / REMOVE / IMPROVE / ADD

Rung 1 - Local corpus

- **WORKING:** 39/40 resolved in the rung-1 micro-proof (p50 260ms wall-clock incl. HTTP overhead); Group A 20/20 in the full T20 run. Zero cost, zero network calls for tire hits.
- **NOT WORKING:** one genuine gap - an EAN-8 code (52454615) sits IN the corpus but codeType does not qualify it for the free-tier resolver, so it fell through to an all-skipped ladder instead of resolving free.
- **TO ADD:** extend isPublicBarcode's free-tier gate to include ean_8, or give the corpus rung its own code-shape-agnostic lookup path so any exact hit resolves free regardless of shape.

Rung 1.5 - Plan D (pre-ladder terminal resolver for public barcodes)

- **WORKING (as intended):** for genuinely unresolved public barcodes, Plan D's free-tier consensus (UPCitemdb + local retail DB + occasional Firecrawl tiebreaker) answers most retail codes before any paid rung runs.

- **NOT WORKING (found and now fixed):** Plan D's terminal design meant Go-UPC/FetchV2/GPT were structurally unreachable for the ENTIRE public-barcode class (upc_a/ean_13/gtin_14) - only 8-digit EAN-8-shaped codes could ever reach the paid ladder. This was the T19 headline finding, closed by T8b (commit 6a800c4, adversarially reviewed, revert-RED proven).
- **TO IMPROVE:** the non-verified Plan D outcome is now stashed and yielded to the ladder rather than returned terminally - reviewed and confirmed correct in the final whole-branch review (Seam 1).

Rung 2 - Go-UPC

- **WORKING:** live-proven post-fix - 9 verified + 1 suggested of 10 known-good GTIN codes, counter delta matched exactly (0->2, then cumulative to 14 across the full T20 dispatch). Non-GTIN (ASIN-shaped) codes correctly gated out before ever reaching the paid client - this is the quota-protection proof the plan required, and it holds.
- **TAIL-RUNG ACCURACY:** 10/10 correct identity or outcome across the T20 run (7 in Group B, 3 in Group C) - the single most reliable paid rung in this session's data.
- **TO IMPROVE:** none found this session; Go-UPC's exact-match behavior and quota gating both held under live load.

Rung 3 - Fetch V2

- **WORKING (live, phase 2):** 20/20 resolved on a rung-isolated sample (Go-UPC and OpenAI keys blanked) vs a 17.6% historical baseline - but see the honesty note below.
- **NOT WORKING / MEASUREMENT CAVEAT:** that 20/20 was a CORPUS result, not a Fetch V2 result - every sampled code actually settled via corpus_exact_barcode in 7-30ms because the "61 double-miss" input set had been fully absorbed into the tire corpus since it was recorded. Fetch V2 itself settled 0 codes and spent 0 Firecrawl credits in that isolated run. A genuine live Fetch V2 proof needs a fresh hard set reconfirmed absent from the corpus at run time.
- **NOT WORKING (known defect class, T20):** search-result-title noise can leak through as a low-confidence suggestion - e.g. a Home Depot bucket code returned "Ryobi stick vacuum clearance deal found - Facebook" via fetchv2 with status:suggested. Never auto-counted, but pollutes Needs Review.
- **TO IMPROVE:** title-relevance filtering / re-ranking before a scraped page title is offered as a suggestion (repeat of a known 2026-07-08 issue, not newly introduced this session).

Rung 4 - GPT-5.5 (probe parity)

- **WORKING:** 9/9 attempted in the T20 hard tail; 6/9 correct identity, all non-auto-counting hallucinations correctly firewalled by the existing status gate (stayed suggested, never verified).
- **NOT WORKING (the severe finding, now fixed - see Section 3):** a bare self-report on a non-public-barcode-shaped code (4-digit part number "1225") auto-verified a completely fabricated product. All 7 gpt_self_report rows in the run shared the identical ungated path; 6 happened to be correct, 1 was not.
- **NOT WORKING (separate, smaller defects, correctly non-auto-counted):** two checksum-valid-but-unassigned UPC-A canaries did not cleanly refuse - one returned a generic "Unidentified item" placeholder after a 23.7s timeout (UX/latency nit), the other fabricated "M23 Signal panel connector housing" (a real hallucination, but stayed suggested). A Pirelli tire code returned "Mediterranean Style Meatloaf Mix" via gpt (suggested, total identity-family swap).
- **TO REMOVE / TO IMPROVE:** the prompt clause driving "never leave productName empty" produces junk suggestions on codes with zero real evidence; a build-time decision (keep / allow-empty / category-flag) is recorded in the spec but not yet implemented. The rung-4 budget math in the original plan was also found arithmetically inconsistent (10-code sample x \$0.39 worst case = \$3.90 > the

\$1.50 sub-cap) - resolved by switching to actuals-based check-before-spend using the route's own gptLadder.spentTodayUsd telemetry.

- **TO ADD (owner-decision item, not yet built):** an EvidenceVerifier-equivalent check that the exact code string appears in fetched source content before trusting ANY GPT verified tier, even for public barcodes where the shape gate does not apply. See Section 6(a).

3. The 1225 Defect: Found, Root-Caused, Fixed, Reverified

3.1 What happened

During the Task 20 live 60-code run, code **1225** - a 4-digit Moen vendor part number ("Moen One-Handle Faucet Replacement Cartridge") with no public barcode structure at all - was auto-verified (status:"verified", corroborationPath:"gpt_self_report") with a completely fabricated identity: *"Spitz Vorosafonya 50%-os gyumolcskeszitmeny 5kg"*, a Hungarian cranberry fruit preparation. This is the single worst outcome the ladder can produce - a wrong-identity auto-count permanently and silently mis-teaches an alias.

3.2 Root cause

src/stores/scanStore.ts's gptTrusted branch (present in both liveDecode and its duplicate in backgroundVerifyDeep) auto-counted any decode with corroborationPath === "gpt_self_report" + status === "verified" + confidence >= 0.8 + a usable product name - with **no check on the scanned code's shape whatsoever**. A GPT self-report is only theoretically falsifiable for a real public barcode (the model claims to have found the exact code on a real page); a 4-digit vendor part number has no public page to have been "found" on, so trusting a bare self-report there is unverifiable by construction. This was a pre-existing design gap (the "GPT LADDER TRUST TIER" owner rule from 2026-07-06), not something newly introduced this session - the earlier T8b reachability fix widened how many codes could reach the already-vulnerable gate, but did not create the gate itself. Two independent gates both had to be absent for this to be unsafe, and neither was present: the store's gptTrusted branch (no shape check) and evidenceScoring.ts's decideAutoVerification ("TRUST-THE-AI" gate, which only blocks codeType=== "vendor_label", never numeric_sku/alpha_sku).

3.3 The fix

Commit **5d810d46b7b3a8957851b4c0c2860430b54c7aa6**, reviewed and confirmed applied correctly. Adds isPublicBarcodeShapeForGptTrust = codeType in [upc_a, ean_13, gtin_14] as the first, required conjunct of gptTrusted in both liveDecode and backgroundVerifyDeep. The existing decodeCorroborated() conjunction (the legitimate app-verified evidence path for corpus/Go-UPC/FetchV2 hits) was NOT touched. TDD evidence: 12/12 tests green including a red-before/green-after pair for exactly code 1225, plus a regression guard proving legitimate upc_a self-report auto-counts are unaffected. Full suite: 1594 pass / 0 fail, tsc clean, eslint clean.

3.4 Graded proof the defect is closed

All 7 codes in the run with corroborationPath "gpt_self_report" reached verified/auto-count status through the identical ungated path - 1225 was simply the one case where the self-report happened to be wrong instead of accidentally correct:

Code	codeType shape	Pre-fix status	This run's identity	Post-fix outcome
28034300	numeric_sku (8-digit)	verified (auto-count)	correct (Falken Wildpeak A/T3W)	needs_review
DCB205	alpha_sku	verified (auto-count)	correct (DeWalt battery)	needs_review
BL1850B	alpha_sku	verified (auto-count)	correct (Makita battery)	needs_review

Code	codeType shape	Pre-fix status	This run's identity	Post-fix outcome
PH7317	alpha_sku	verified (auto-count)	correct (FRAM oil filter)	needs_review
K060841	alpha_sku	verified (auto-count)	correct (Gates belt)	needs_review
1225	numeric_sku	verified (auto-count)	WRONG (fabricated Hungarian cranberry product)	needs_review
GP1043211	alpha_sku	verified (auto-count)	correct (Kohler sprayhead)	needs_review

6 of 7 "got lucky" this run (correct answer despite an unverifiable, ungated trust path); 1225 proves the path was never actually safe. The fix demotes all 7 uniformly - a deliberate, symmetric trade of a small recall cost (6 previously-correct auto-counts become suggestions requiring human approval) for closing the entire hallucination-auto-count hole for this code-shape class, matching CLAUDE.md's resolver-trust rule: "Unknown is ACCEPTABLE. Prefer Needs Review over a wrong guess."

Auto-count precision - the headline safety number

	Total auto-counted (status=verified)	Correct	Wrong	Precision
Pre-fix	32	31	1 (code 1225)	96.9%
Post-fix	25	25	0	100%

(32 pre-fix = 20 Group A corpus hits + 5 Group B single-source/Go-UPC exact hits + 7 Group C gpt_self_report hits. Post-fix, all 7 gpt_self_report hits demote to needs_review, leaving 25 auto-counts, all correct. Graded offline by claude-sonnet-5 directly, per owner order that mechanical grading be done by a lower-tier model with no escalation to opus.)

4. Discount Tire Harvest

4.1 Pipeline

- Sitemap discovery: `sitemaps.discounttire.com/sitemap_full_product.xml` -> one .gz child, 18,950 URLs, of which 7,886 are tire product pages (product URLs are `/buy-tires/<slug>/p/<digits>`).
- Deterministic JSON-LD tire parser, with the finding that DT's JSON-LD Product block has NO GTIN - the barcode and full specs arrive via the page's own `webapi/discounttire.graph?op=productByCode` response, captured from natural page traffic (no extra requests needed).
- Poison guard: check-digit validation + brand-prefix firewall rejects any row whose "gtin" field is actually DT's internal article code (not a real GS1 barcode) rather than a genuine barcode.
- Playwright fetcher with a rolling block-rate stop: sequential fetches, 2000-4000ms randomized delay, realistic Chrome-on-Windows context, per-worker 30% block-rate hard stop, host allowlist `www.discounttire.com`.
- Cross-source-safe merge: new GTINs are added; codes already present from another source are left as-is (no overwrite of existing corpus provenance).

4.2 Pilot (100 pages, 2026-07-08) - PASS

Metric	Gate	Result
Block rate	$\leq 30\%$	0.0% (0 blocked / 100)
Valid-GTIN rows	≥ 60	96
Errors / parse misses	-	0 / 0
Guard rejections	-	4 (all correct - DT internal article codes, not real GTINs)
Throughput	-	~733 pages/hour (8m11s wall)

4.3 Fleet - final numbers (full catalog)

Metric	Value
Guarded rows harvested	7,486 across the full catalog
Blocks across the entire run	0
Poison-guard rejects	221
New tire GTINs merged into corpus	+2,029 (5,553 cross-source dupes kept as-is)
Corpus DB	rebuilt (342 MB); corpus JSON committed, .db and .bak gitignored
Spot-check proof	20/20 PASS
Integration proof	10/10 new DT GTINs resolved via <code>resolveExactBarcode</code> (rung-1 function) -> <code>corpus_exact_l</code>
Resilience	Detached worker fleet (Start-Process, separate PIDs) survived 2 EPERM crashes mid-run (

4.4 Spend

\$0. The harvest is compute-only (Playwright against DT's own public pages plus natural page traffic already returned by the site) - no paid API calls of any kind.

5. Spend Reconciliation (Cost-Truth Rule)

Per the owner's Paid API Cost Truth rule: every figure below is a **computed floor** from response metadata and route-recorded/file-backed actuals. True spend must be reconciled against each provider's own billing console before being quoted as final.

Provider	This session's computed floor	Cap	Notes
Go-UPC	14 lookups total (Turso goupc_usage counter, month 2026-07: 00/month phase cap	Atomic in-SQL increment (used = used + .	
GPT-5.5	\$1.1309 total, 13 calls (file-backed .gpt-ladder-usage.json, date 2026/07/09)	compute-enforcedGptLadderBudget() gates before ev	
Firecrawl	<= ~112 credits worst-case reserved across all proof runs today (02-400 credits 2 Pl	API phase cap head cap 100 is exposed by	
Discount Tire harvest	\$0	n/a	Compute-only; no paid API calls.
Grading pass (this report's source data)	\$0	n/a	Offline grading over already-recorded JSC

Wallet line: computed floor - Go-UPC 14 lookups, GPT \$1.13 (persisted file actuals), Firecrawl <= ~112 credits worst-case reserved; DT harvest \$0. True spend = provider consoles (Go-UPC / OpenAI / Firecrawl). No run was stopped early by a spend cap in the final clean attempt.

6. Open Items / Owner Decisions

Five items require an explicit owner decision or are flagged as known, deliberately-unfixed gaps. None are blocking; all are surfaced per the doctrine's no-partial-completion rule.

(a) EvidenceVerifier on public-barcode GPT self-reports

The 1225 fix closes the hole for non-public-shaped codes (vendor/SKU/part numbers), but a real public UPC/EAN/GTIN GPT self-report can still be a hallucination with a fabricated source URL - the shape gate only blocks vendor/short codes, not a wrong-product claim on a VALID public barcode. Adding an independent EvidenceVerifier-style check (confirming the exact code string appears in fetched source content) would close this, but would contradict the owner's 2026-07-06 "probe parity / no app-side questioning" order (raw code in, answer taken as returned). Current posture - trust a GPT self-report on a real public barcode at confidence ≥ 0.8 - is the owner-approved default. This is a deliberate, owner-approved trade-off being resurfaced for revisit, not treated as a defect.

(b) FetchV2/GPT raw-archive module never built

The plan's Task 2 "raw decode archive" for Fetch V2 and GPT-5.5 raw responses does not exist as a standalone module. Only the Go-UPC rung's appendArchive call is wired today (backed by the Turso decode_archive table, 1-in-200 hit sampling by design). The T20 run recorded 0 archive entries total, which is the CORRECT designed outcome given only 10 Go-UPC hits occurred (below the 200-hit sampling threshold) - but it means Fetch V2 and GPT raw responses are not currently captured anywhere for later audit.

(c) All repo hard-set test pools are now corpus-absorbed

Every pre-existing "known-miss" hard-set pool in the repo (tmp-atrisk-codes.json, all 10 tmp-loop*-codes.txt files, tmp-goupc-200-misses.txt, tmp-fetchv2-misses.txt - 74 + 61 codes) was checked during this session and found 100% absorbed into one of the two local corpora (mostly via the DT harvest and prior corpus growth). Future live-ladder proofs cannot reuse these lists to exercise the paid rungs - they need freshly synthesized codes verified absent from both corpora at run time, as this session's Group B/C did.

(d) FetchV2 junk-title suggestion class

Fetch V2 can surface an irrelevant scraped page title as a low-confidence suggestion (e.g. "Ryobi stick vacuum clearance deal found - Facebook" for a Home Depot bucket code). Never auto-counted, but it pollutes the Needs Review queue with noise. A repeat of a known 2026-07-08 issue, not newly introduced. No fix built this session; flagged for a future title-relevance pass.

(e) Prefix-floor placeholder-name collision on OLD pre-field persisted reviews

Found in the final adversarial whole-branch review (Important, low likelihood). After a customer reload strips identifier fields, the reload-resilient fallback re-identifies a scan's own placeholder by name. A prefix-floor placeholder name is brand-only, not code-specific ("**<Brand>** / product unconfirmed") - if a customer scans two different unresolved codes that share a GS1 prefix-derived brand, both placeholders get the identical name, and a reload can cause the wrong one to be matched on resolve. Net total count is preserved (no double-count or lost count), but attribution can be wrong and a stale row can survive. New records (post the STABLE-ID fix, commit ba3e187, which added a provisionalProductId field) are immune - id-based matching is tried first, and the name-fallback only applies to pre-field persisted reviews from before that fix landed. Recommended fix: scope the name fallback to at most one candidate, or skip it entirely when more than one counted product shares that

exact name. Not a merge blocker; should land before multi-code prefix-floor scans become common.

7. Known Limitations / Deferred Follow-ups

- **Rung 3 (Fetch V2) has no fresh live proof** against a genuinely corpus-absent hard set - the phase-2 rung isolation run resolved 20/20 via the corpus short-circuit, not via Fetch V2 itself (0 Fetch V2 settles, 0 Firecrawl credits actually spent in that run). A real live Fetch V2 proof needs a newly synthesized hard set.
- **evidenceScoring.ts's vendorCodeNoAlias gate** only fires for codeType==="vendor_label", not numeric_sku/alpha_sku/messy. Currently non-exploitable because it is always ANDed with evidenceGatePassed in the only production call sites (both traced and confirmed) - but it is under-inclusive defense-in-depth. Left unfixed this session as explicitly out of scope for the minimal 1225 fix; flagged as a follow-up.
- **debug.aiCalled is true whenever the GPT rung merely ran**, not only when it actually contributed the winning answer - a pre-existing diagnostic looseness, no behavior impact, noted in both the T8b and final whole-branch reviews.
- **Turso decode_archive schema divergence and month-row accumulation** - accepted in a prior review; the archive is write-only/inert today and does not affect correctness.
- **No stable provisionalProductId on pre-fix persisted reviews** (see Section 6e) - the deferred "stable provisional product id" follow-up would close both the identifier-strip fragility and the prefix-floor name collision cleanly for all records if implemented as a backfill/migration.
- **Session required real repair loops to complete cleanly** - the Task 20 full-ladder run took 4 attempts (server-boot timeout, an exhausted daily call-volume test guard, and a mid-run dev-server crash) before producing a clean 60/60 result with 0 skipped rows. None of these touched real provider spend; all are documented in scripts/proof-ladder-report.md and fixed in the committed proof script (scripts/proof-full-ladder.mjs, consecutive-fetch-failure circuit breaker added) for future reruns.
- **Binding qa:revision gate reached GREEN only after two earlier partial runs** - an earlier attempt showed 8/12 QA bots failing (4 real failures traced to shared decode-cache/Turso state polluted by concurrent live proof runs on the same branch, not a regression in the session's own code changes); the final clean run (after the harvest fleet quieted the machine) passed 12/12.
- **Not pushed, not deployed.** All work in this report lives on feat/decode-ladder-goupc, 128 commits ahead of master, unpushed. Merge/PR strategy remains an explicit owner decision (finishing-a-development-branch options pending presentation).

Source artifacts synthesized into this report: scripts/proof-ladder-report.md, scripts/proof-full-ladder-graded.json, scripts/proof-full-ladder-results.json, .superpowers/sdd/task-T20GRADE-report.md, .superpowers/sdd/task-1225-report.md, .superpowers/sdd/task-1225-fix-report.md, .superpowers/sdd/task-FINALREVIEW-report.md, scripts/dt-harvest/PILOT-2026-07-08.md, .superpowers/sdd/progress.md.